

# Compiling with Counterexamples:

correctness, copying, cost and comprehension for partial compilers

Valeria de Paiva

October 7, 2026

## Abstract

A partial compiler — one that reduces a source program, delegates, and assembles the target from whatever comes back — is a Dialectica morphism with its relation discarded. Restoring the relation, read as a relation of refutation and valued in a lineale rather than in the booleans, gives compositional correctness at no cost in the data: a pass preserves correctness exactly when it is a morphism, so being correct is a condition a partial compiler either meets or fails, never extra data it has to carry, and a stack of passes is verified by pushing the specification of the target back through the passes one at a time, rather than by proving anything about the composite. Two further consequences follow by unfolding definitions. Whether a partial compiler may duplicate its source is a question about the target type alone, which accounts for a failure of associativity reported when compile-time effects are moved inside. Whether it can carry a cost model is settled by which equations that type satisfies. Restricting to the sources for which the specification holds gives a comprehension, with which we conjecture that the verified partial compilers form an object rather than a class, right adjoint to the obvious product.

## 1 Partial compilers

A compiler need not do the whole job. It may perform one pass — lowering a loop, partitioning a computation across machines, choosing a representation — and leave the rest to somebody else. To describe that, Budiou, Galenson and Plotkin [1] introduce the *partial compiler*: given a source program it produces a reduced source, to be handed on, together with a way of assembling the final target from whatever target comes back:

$$f : \text{source} \rightarrow \text{source}', \quad F : \text{source} \times \text{target}' \rightarrow \text{target}. \quad (1)$$

Both halves are needed, and the first argument of  $F$  is the point of the definition: assembling the final target generally requires the original source, not merely the target returned, since the material the pass set aside has to be put back.

Partial compilers plug into one another. Handing the reduced source of one to another and assembling in the reverse order gives

$$(f, F); (g, G) = (g \circ f, (u, z) \mapsto F(u, G(f(u), z))), \quad (2)$$

a partial compiler again, and the operation is associative with an evident identity. So there is a category, whose objects are pairs  $(\text{source}, \text{target})$  — or, in the vocabulary those authors use, pairs of a *problem* and a type of *solutions* to it.

Composition is not the only operation they define. There is a tensor, running two partial compilers side by side; a star, applying one to each element of a list; conditionals and case analysis; and a *functor* operation  $\text{PCFunc}(f, g)$ , assembled from a map  $f$  on sources and a map  $g$  on targets, whose generation function applies  $g$  and ignores the source. Only the tensor and this last are needed below.

Their vocabulary is not accidental. Budiu, Galenson and Plotkin trace partial compilers to categorical logic and to theorem proving: to the dialectica categories of de Paiva and Hyland [3], and to Milner’s tactics, which they show are partial compilers whose reduced source is a list of subgoals. The shape is older than either. Reducing a problem to another problem, and assembling a solution of the first from a solution of the second, is Kolmogorov’s reading of intuitionistic logic as a calculus of problems, and Kolmogorov’s problems, Veloso’s abstract problems and Blass’s questions and answers are related to one another precisely by Dialectica constructions [10]. So the construction that organises the notions of problem is the one applied to compilers here, and what follows applies verbatim to partial compilers, tactics and problems alike. Nothing in the definition of a partial compiler says that it is correct, and supplying that is what the relation of the Dialectica construction is for.

## 2 The Dialectica construction

Fix a cartesian closed  $\mathcal{C}$  with finite coproducts. Gödel’s Dialectica interpretation assigns to a formula  $A$  a formula  $\exists u \forall x \alpha(u, x)$  with  $\alpha$  quantifier-free, where  $u$  ranges over *witnesses* — proposed evidence for  $A$  — and  $x$  over *counterexamples*, challenges to a proposed witness. Proving an implication yields, beyond a map on witnesses, a way of turning a challenge to the conclusion into a challenge to the premiss.

The categorical construction takes that shape as its objects. Let  $L$  be a *lineale*: an object of truth values carrying an order  $\sqsubseteq$ , a symmetric monoidal  $\otimes$  with unit  $e$  that is monotone in each argument, and a right adjoint  $\multimap$  to it. The two-element boolean algebra, with  $\otimes = \wedge$ , is the classical case. An object of  $\text{Dial}_L(\mathcal{C})$  is a triple  $(U, X, \alpha)$  with  $\alpha: U \times X \rightarrow L$ ; a morphism to  $(V, Y, \beta)$  is

$$f: U \rightarrow V, \quad F: U \times Y \rightarrow X, \quad \text{with} \quad \alpha(u, F(u, y)) \sqsubseteq \beta(f(u), y). \quad (3)$$

Witnesses go forwards and counterexamples backwards. What the condition means depends on what  $\alpha$  is taken to say, and §4 fixes that for compilers. We write  $\ell = (f, F)$  for such a pair, and call it a morphism or, when the compiler reading is meant, a pass.

The construction is the original dialectica category of de Paiva [2, 3]. It is symmetric monoidal closed, with tensor computed coordinatewise,

$$(U, X, \alpha) \otimes (V, Y, \beta) = (U \times V, X \times Y, ((u, v), (x, y)) \mapsto \alpha(u, x) \otimes \beta(v, y)), \quad (4)$$

and unit  $I = (\mathbf{1}, \mathbf{1}, e)$ . A reader arriving from Chu spaces or from optics will expect the function spaces  $X^V \times Y^U$  in the second coordinate. Those belong to the variant  $\text{Gir}_L(\mathcal{C})$  described below, where the backward map does not see the witness; once  $F$  takes  $U$  as an argument the exponentials are already absorbed, as currying  $\text{Hom}(A \otimes B, C) \cong \text{Hom}(A, B \multimap C)$  shows. The distinction matters here, because §5 turns on duplication being a question

about  $X$  alone, which is what the coordinatewise tensor says and the crossed one denies. The cartesian product, which §7 needs, sums in the second coordinate instead:

$$(U, X, \alpha) \times (V, Y, \beta) = (U \times V, X + Y, ((u, v), \text{inl } x) \mapsto \alpha(u, x), ((u, v), \text{inr } y) \mapsto \beta(v, y)), \quad (5)$$

a challenge to a pair being a challenge to one component or to the other.

Two further pieces of the construction are needed below. The first is the variant  $\text{Gir}_L(\mathcal{C})$ : the same objects, but a morphism  $(U, X, \alpha) \rightarrow (V, Y, \beta)$  is

$$f: U \rightarrow V, \quad F: Y \rightarrow X, \quad \text{with} \quad \alpha(u, F(y)) \sqsubseteq \beta(f(u), y), \quad (6)$$

the backward map now independent of the witness. It is the fragment in which nothing about the source flows backwards, and §3 identifies one of their operations on partial compilers with it. The second is the modality:  $\text{Dial}_L(\mathcal{C})$  carries a comonad  $!$  which replaces the counterexample object  $X$  by the free commutative monoid on it. Proposition 3 says which objects already have the structure that  $!$  supplies freely.

### 3 Partial compilers are Dialectica morphisms

Displays (1) and (3) are the same data, and switching the relation off makes the condition of (3) vacuous. But the agreement is not only at the level of the data: the categorical structure matches item by item.

**Proposition 1.** *For  $L = \mathbf{1}$  the condition in (3) holds always, and  $\text{Dial}_1(\mathcal{C})$  is isomorphic to the category of partial compilers of Budiú, Galenson and Plotkin by the isomorphism that is the identity on data: an object, whose relation is the unique map into  $\mathbf{1}$ , is a pair (source, target), a morphism  $(f, F)$  is a reduction together with a generation function, and (2) is Dialectica composition.*

The identity  $(\text{id}_U, \pi_2)$  is their identity partial compiler, their tensor is (4), and their functor operation is the inclusion of  $\text{Gir}_1(\mathcal{C})$ , in which the backward map ignores the source. Those authors say as much from their side: for the identity monad their category of tactics is equivalent to the dialectica category. The identification was set out in a talk on their preprint in 2011 [7]. So a partial compiler is a dialectica morphism whose relation has been thrown away, and the four sections that follow say what restoring it gives back: correctness, a criterion for copying, a cost model, and, conjecturally, a type of verified passes.

The same data is also a *bimorphic lens* [8, 9]:  $\text{get}: U \rightarrow V$  and  $\text{put}: U \times Y \rightarrow X$ , the abstraction of a field of a record, with  $\text{put}$  needing the old state because everything outside the field must be carried along. Monomorphic lenses,  $U = X = S$  and  $V = Y = A$ , are the familiar case. The compiler reading is the one used here, because it is the one with an external correctness criterion attached: a target is right or wrong for a source by comparison with a semantics, whereas the lens laws are coherence between the two directions rather than agreement with a specification. That dialectica categories and lenses are two views of one construction is by now well understood; the observation that lenses are dialectica morphisms appears in Hedges. What seems not to have been said is that partial compilers belong on that list: Budiú, Galenson and Plotkin arrive at partial compilers from the dialectica categories and from Milner’s tactics, and do not mention lenses.

## 4 Correctness

Read  $\alpha(S, T)$  as saying that  $T$  is a *wrong* implementation of  $S$ : a refutation relation, with  $a \sqsubseteq b$  read as “ $a$  is no more wrong than  $b$ ”. Nothing in the construction forces this, and in Gödel’s own reading the relation is affirmative,  $\alpha(u, x)$  saying that the witness  $u$  withstands the challenge  $x$ . But a partial compiler puts the target in the place of the challenge, and a target is not adversarial: a correct target is a failed challenge to its source, not a successful one. Under the refutation reading condition (3) says:

if the target assembled for  $S$  is wrong, then the target returned for the reduced source  $f(S)$  was already wrong.

Contrapositively, a pass whose child returns a correct target assembles a correct one. That is compositional correctness for a partial compiler, on the nose and with no reformulation: the pass preserves correctness, which is to say that it introduces no errors of its own.

The affirmative reading, with  $\alpha(S, T)$  saying that  $T$  is correct for  $S$ , turns the same condition into its converse. A correct assembled target can then only come from a correct returned one: the pass reflects correctness and never repairs an error, but nothing stops it introducing one. A pass that ignores what comes back and always emits garbage is a morphism under the affirmative reading, its condition holding vacuously, while a pass that ignores what comes back and always emits a target known to be correct is not; the refutation reading classifies both the right way round. Complementing the lineale does not reconcile the two readings. From an affirmative specification  $\alpha$  one obtains the refutation specification  $\neg\alpha$ , but negation reverses  $\sqsubseteq$ , so the pair  $(f, F)$  is a morphism from  $(U, X, \neg\alpha)$  to  $(V, Y, \neg\beta)$  exactly when  $\beta(f(u), y) \sqsubseteq \alpha(u, F(u, y))$ , which is (3) reversed. Complementation exchanges preservation and reflection rather than identifying them.

A compiler that delegates nothing is the degenerate case, and it is where the word “correct” acquires a definition. Suppose  $L$  has a least element  $\perp$  and write  $\mathbf{0} = (\mathbf{1}, \mathbf{1}, \perp)$ . A morphism  $(U, X, \alpha) \rightarrow \mathbf{0}$  consists of the unique map  $U \rightarrow \mathbf{1}$  together with a backward map  $U \times \mathbf{1} \rightarrow X$  — that is, nothing but a compiler  $C: \text{source} \rightarrow \text{target}$  — and its condition reads

$$\alpha(u, C(u)) \sqsubseteq \perp, \tag{7}$$

that the target  $C$  assigns to a source is never a counterexample to it. Under the affirmative reading the same condition would say that  $C$  never produces a correct target. So a correct compiler is not a compiler with a hypothesis attached: it is a morphism, and an arbitrary compiler is merely a map of  $\mathcal{C}$  that fails to be one. Replacing  $\perp$  by an arbitrary  $\delta \in L$  gives the compilers whose refutations are bounded by  $\delta$ , which §7 uses once §6 has made  $L$  quantitative. The tensor unit  $I = (\mathbf{1}, \mathbf{1}, e)$  is the wrong object here: when  $e$  is the top of  $L$  the condition  $\sqsubseteq e$  is satisfied by everything.

Suppose now that a pass delegates to a child, and that we know what specification the child’s output must meet. What must the pass’s own source end satisfy? There is a best answer, and it is computed rather than guessed at.

Let  $\ell = (f, F)$  be a pass from  $(U, X)$  to  $(V, Y)$ , and let  $L$  be complete. The child’s specification is a relation  $\beta$  on  $V \times Y$ ; we are looking for a relation on  $U \times X$ , so we must say, for a source  $u$  and a candidate counterexample  $x$  to it, how wrong  $x$  is allowed to be. Now  $\ell$  only ever produces the counterexample  $x$  by assembling it from something the child

returned: the counterexamples  $y$  that  $\ell$  turns into  $x$  at  $u$  are exactly those with  $F(u, y) = x$ . Condition (3) bounds  $\alpha(u, x)$  by  $\beta(f(u), y)$  for each of them, so  $x$  may be no more wrong for  $u$  than the least wrong of those  $y$  is for  $f(u)$ , and we set

$$\ell^\# \beta(u, x) = \bigwedge \{ \beta(f(u), y) : F(u, y) = x \}, \quad (8)$$

taking the top of  $L$  when the set is empty:  $\ell$  never produces  $x$  at  $u$ , and so places no constraint on it. In the boolean case (8) says:  $x$  refutes  $u$  only if every  $y$  that  $\ell$  would turn into  $x$  already refutes  $f(u)$ . Transporting in the other direction instead,

$$\ell_! \alpha(v, y) = \bigvee \{ \alpha(u, F(u, y)) : f(u) = v \} \quad (9)$$

collects what a specification on the source end forces at the child's.

**Proposition 2.** *The two transports are adjoint,  $\ell_! \dashv \ell^\#$ , and  $\ell$  is a morphism  $(U, X, \alpha) \rightarrow (V, Y, \beta)$  if and only if  $\alpha \sqsubseteq \ell^\# \beta$ , equivalently  $\ell_! \alpha \sqsubseteq \beta$ . Moreover  $(\ell' \ell)^\# = \ell^\# \ell'^\#$ .<sup>1</sup>*

So  $\ell^\# \beta$  is the weakest specification a source end can carry while leaving  $\ell$  correct with respect to  $\beta$  — a weakest precondition, with (3) as the Hoare triple for a program that runs in two directions. The equation  $(\ell' \ell)^\# = \ell^\# \ell'^\#$  is what makes this useful. To verify a stack  $\ell_1; \dots; \ell_n$  against a specification at the far end, push that specification back through the passes one at a time and check each pass against whatever arrives at it; nothing has to be proved about the composite. And nothing about the compilers changes to allow this: no new composition operator, no extra data for a pass to carry.

The relation was not dropped and then never missed. Among the operations Budi, Galenson and Plotkin define is a conditional, and the use they put it to is patching a compiler one cannot recompile. Assume a predicate *bug*: *source*  $\rightarrow$  **bool** describing the sources — or a superset of them — on which an optimising compiler  $C_{\text{opt}}$  produces an incorrect target, and a simple compiler  $C_{\text{simple}}$  that is always correct; then

$$\text{IF } \textit{bug} \text{ THEN } C_{\text{simple}} \text{ ELSE } C_{\text{opt}} \quad (10)$$

hides the bug. But *bug* is  $\alpha$  evaluated along a compiler: *bug*( $S$ ) asserts  $\alpha(S, C_{\text{opt}}(S))$ , and the hypothesis that  $C_{\text{simple}}$  is always correct is exactly the condition (7) for it to be a morphism into  $\mathbf{0}$ . With no relation between sources and targets available, the predicate has to live on the source type alone, and a superset is the best that can then be managed, where the exact set is a comprehension. Restoring  $\alpha$  makes the proposal a statement with an obligation attached, and Proposition 2 says what the obligation of the composite is.

## 5 Copying and compile-time effects

They report a failure whose cause the construction here makes visible. Moving compile-time effects inside, so that  $f: P \rightarrow T(P')$  and  $g: P \times S' \rightarrow T(S)$ , does not give a category: composition fails to be associative. Their fix is to keep a single monad outside. The diagnosis

<sup>1</sup>In fibrational language: the functor  $\text{Dia}_L(\mathcal{C}) \rightarrow \text{Dia}_1(\mathcal{C})$  that forgets the relation is a bifibration with posetal fibres. The base is the category of §3, the fibre over  $(U, X)$  is the poset of relations  $U \times X \rightarrow L$  ordered pointwise, reindexing along  $\ell$  is  $\ell^\#$ , and  $\ell_!$  is its left adjoint. Nothing below uses this.

is visible in their own syntax, where the source is bound in both the reduction and the generation clause: it is used twice. Composition (2) makes this concrete, since the composite recomputes  $f(u)$  inside its generation clause in order to call  $G$ ; with effects inside, the two bracketings of a triple composite perform the effect of the first reduction twice and once respectively. Duplication is free in a cartesian setting and not free otherwise, and the general statement is the following.

**Proposition 3.** *A commutative comonoid in  $\text{Dial}_L(\mathcal{C})$  is a comonoid in the first coordinate together with a commutative monoid  $(\emptyset, \star)$  on the second, subject to*

$$\alpha(u, \emptyset) \sqsubseteq e \quad \text{and} \quad \alpha(u, x \star x') \sqsubseteq \alpha(u, x) \otimes \alpha(u, x'). \quad (11)$$

*Proof.* Weakening  $A \rightarrow I$  has backward map  $U \times 1 \rightarrow X$ , that is a point  $\emptyset$  of  $X$ ; contraction  $A \rightarrow A \otimes A$  has backward map  $U \times (X \times X) \rightarrow X$  by (4), that is a multiplication  $\star$  on  $X$ . Instantiating the morphism condition (3) for these two maps gives exactly (11), and the comonoid laws for  $A$  become the monoid laws for  $(\emptyset, \star)$ : counit gives unitality, coassociativity gives associativity, and cocommutativity gives commutativity. Both backward maps take  $u$  as an argument, so what one obtains is strictly a  $U$ -indexed family of monoid structures on  $X$ ; we write  $\emptyset$  and  $\star$  for the family and ask the laws to hold uniformly in  $u$ .  $\square$

The variance turns the comonoid into a monoid, and the first coordinate costs nothing. So whether a source may be duplicated is a question about the *target* type and nothing else: it must absorb two targets into one.<sup>2</sup> That is what fails when effects move inside, and it is what a single outer monad restores.

Their CASES operation shows the same dependence on the target type from the other side. It combines compilers sharing a target type, and they note that categorical sums exist when the solution objects are the same,  $(P_1, S) + (P_2, S) = (P_1 + P_2, S)$ . That restriction is not a convenience but exactly the hypothesis under which the sum exists. For arbitrary  $(U_1, X_1, \alpha_1)$  and  $(U_2, X_2, \alpha_2)$  the candidate is  $(U_1 + U_2, X_1 \times X_2)$ , and a mediating map into a cocone is not unique, since each injection constrains only one component of the backward map: one gets a weak coproduct only. With  $X_1 = X_2 = X$  that pairing never arises. The injections are  $(\text{inl}, \pi_2)$  and  $(\text{inr}, \pi_2)$ ; given a cocone  $(g_i, G_i): (U_i, X, \alpha_i) \rightarrow (Q, R, \gamma)$ , the mediator must be  $[g_1, g_2]$  forwards and, since coproducts in a cartesian closed  $\mathcal{C}$  are distributive,  $[G_1, G_2]$  backwards, uniquely; and (3) holds summand by summand for  $[\alpha_1, \alpha_2]$ . So  $(U_1 + U_2, X, [\alpha_1, \alpha_2])$  is a genuine coproduct.

**Remark 4** (order). Contraction is symmetric, so Proposition 3 asks for a *commutative* monoid, and a target type that assembles its two arguments in a fixed order does not qualify. Their own sequential partial compiler is such a type: it splits a source into a prefix and a suffix and generates the composite target from the two it receives, in that order. So the pass that motivates their star operation is one whose sources cannot be duplicated, and whether the right notion here is a commutative monoid or a monoid — that is, whether the copying

---

<sup>2</sup>In the boolean case, with  $\otimes = \wedge$ , the second condition in (11) says that the combined target  $x \star x'$  refutes  $u$  only if both  $x$  and  $x'$  do, and (12) says likewise that a list of targets is refuted only if every entry is. Read affirmatively these would be the expected statements, that a combination is correct only if its parts are; read as refutations they say instead that one good part is enough, which suits a pass choosing between two returned targets better than one merging them, and suits a list not at all. The reading that (3) forces in §4 and the reading the monoidal structure suggests therefore come apart here, and we leave them apart.

available to a pass should be sensitive to the order in which its children’s targets are assembled — is left open.

The construction has a second reading, in which  $U, X$  are the states and  $V, Y$  the views of a bimorphic lens, and there Proposition 3 accounts for the hierarchy of optics: which of lens, traversal and affine traversal admits copying is decided by the monoid available on the update type. That is a separate story, told elsewhere.

## 6 Cost

The monoid on the target type and the  $\otimes$  of the lineale are not independent: an equation satisfied by the first is inherited by the second along the image of  $\alpha$ . Suppose  $\star$  is idempotent,  $x \star x = x$ . Then (11) gives

$$\alpha(u, x) \sqsubseteq \alpha(u, x) \otimes \alpha(u, x),$$

so every value  $\alpha$  takes lies below its own square. If the lineale is affine —  $e$  the top, so that  $a \otimes b \sqsubseteq a \wedge b$  — the reverse inequality holds as well and  $\alpha$  takes values among the  $\otimes$ -idempotents; if all of  $L$  is idempotent then  $\otimes = \wedge$  and the lineale is cartesian. Over the Lawvere quantale, described below, the same computation reads  $a \geq a + a$ , forcing  $a = 0$  or  $a = \infty$ : an idempotent target type makes every target either refuted outright or not at all, and the quantitative structure collapses to its boolean part. A free monoid on the target type imposes no equation at all, and leaves the lineale unconstrained.

Where the lineale is quantitative, the specification measures how far a target is from being wrong. The Lawvere quantale is  $[0, \infty]$  with the order *reversed*,  $\otimes = +$  and  $e = 0$ ; relations valued in it are the generalised metric spaces of [4], where the reversal is what lets a  $L$ -valued relation be read as a distance and  $\otimes$  as the triangle inequality. Under the reading of §4 its top 0 is outright refutation and its bottom  $\infty$  is no refutation at all, so  $\alpha(S, T)$  is the margin by which  $T$  clears failure for  $S$ : the slack left under a bound on the size of  $T$ , or its running time, or its register pressure, rather than the size or time itself. Cost read directly would need  $[0, \infty]$  in its usual order, where  $+$  has no right adjoint, since  $a + 0 = a$  and so  $\otimes$  does not preserve the least element. Take  $\alpha(S, T) = B - c(T)$ , the slack under a bound  $B$  on a cost  $c$ , and  $\beta$  likewise under a bound  $B'$ . For targets within their bounds, condition (3) then says that a pass adds at most  $B - B'$  to the cost of the target its child returned, and Proposition 2 computes the bound a stack needs from the bounds of its passes. Specifications of this shape, with equations holding up to a numerical distance rather than on the nose, are the subject of quantitative algebraic reasoning [6]; the related discipline of tracking a resource annotation in the types themselves is quantitative type theory [5]. This is the setting their multi-machine partitioning examples ask for, and it is available at no extra cost provided the target type is free.

Their star operation supplies the example that makes the point. A compiler is lifted to lists of sources by applying it pointwise, and to carry a specification along with it one needs a relation on lists — that is, a way of scoring a list of values of  $L$ . One is already to hand, since  $(L, \otimes, e)$  is a monoid:

$$\alpha^*(\vec{u}, \vec{x}) = \bigotimes_i \alpha(u_i, x_i), \quad (12)$$

with  $e$  on the empty list. Over the Lawvere quantale (12) is the total margin of a list of targets. Note where the free monoid has gone: in §5 it was a structure asked of the target

type, and here it is the aggregation of a list of values of  $L$ . The two are the same construction on opposite sides of the relation, and the second is what makes the sequential partial compiler of Remark 4 carry a cost model even though it cannot copy.

## 7 The type of verified partial compilers

Nothing on the target side can restrict the source: a specification cuts down which sources a pass is meant to handle, and that is an operation on the first coordinate. Under the reading fixed in §4, a source meets its specification when nothing refutes it, so for  $\delta \in L$  set

$$\mathbf{Cmp}_\delta(U, X, \alpha) = (\{u \mid \bigvee_x \alpha(u, x) \sqsubseteq \delta\}, X, \alpha), \quad (13)$$

the object whose sources are refuted by nothing worse than  $\delta$ , together with the inclusion  $(i, \pi_2): \mathbf{Cmp}_\delta(U, X, \alpha) \rightarrow (U, X, \alpha)$ , which is a morphism. Taking  $\delta = \perp$  gives the sources with no counterexample at all, and  $\mathbf{Cmp}_\perp(U, X, \alpha)$  is the object whose witnesses are exactly those  $u$  for which every compiler into  $\mathbf{0}$  out of  $u$  is correct, in the sense of (7). Over the Lawvere quantale  $\perp$  is  $\infty$ , and a general  $\delta$  is a required margin:  $\mathbf{Cmp}_\delta$  keeps the sources every target of which clears failure by at least  $\delta$ .

The construction is defined on objects only. To extend it to a pass  $\ell = (f, F): A \rightarrow B$  one needs  $f$  to carry sources refuted by nothing worse than  $\delta$  to sources of the same kind, and condition (3) does not give this: it bounds  $\alpha(u, F(u, y))$  by  $\beta(f(u), y)$ , and so says nothing about how small  $\beta(f(u), y)$  is. What would give it is the converse inequality  $\beta(f(u), y) \sqsubseteq \alpha(u, F(u, y))$ , the reflection of correctness that §4 set aside. On the passes satisfying both inequalities, that is with equality in (3), the construction is an idempotent comonad with the inclusion as counit, but that is a strong restriction. Preserving correctness constrains what a pass does to targets, while keeping sources verified asks for the converse, so the two readings of §4 meet again here. The affirmative form of the definition,  $\delta \sqsubseteq \bigwedge_x \alpha(u, x)$ , is functorial on all passes, since  $\delta \sqsubseteq \alpha(u, F(u, y)) \sqsubseteq \beta(f(u), y)$ , but under the reading of §4 it selects the sources that every target refutes, which is not what is wanted.

Suppose now that the target type carries the error monad  $X \mapsto X + 1$ , so that a pass may decline to produce a target. This is not an extra assumption so much as an acknowledgement of practice: a pass that cannot handle a source has to say so.

**Conjecture 5.** With targets in  $X + 1$ , the object  $[A, B]$  of *verified* partial compilers from  $A$  to  $B$  exists, and is right adjoint to  $- \times A$ , for the product of (5), whose target type is  $X + Y$ .

The witness object of  $[A, B]$  would be cut down to those pairs that satisfy their own condition, making it the type of passes that meet the specification rather than a type of candidates with a side condition attached; and the adjunction would be cartesian rather than monoidal. The option of declining is what lets a pair survive the comprehension, since without it the object collapses to the totally verified passes — which is the same structure Proposition 3 says blocks copying. A pass that may fail cannot duplicate its source. We state this as a conjecture rather than a theorem because it is the one claim here that is not settled by unfolding a definition, and the failure of functoriality above is part of the reason: the comprehension it needs is a comonad only on a subcategory.

**Remark 6.** Any such adjunction is likely to hold only up to identification of morphisms that agree wherever their conditions are not vacuous. Whether to quotient is a decision about which category is under study; it is not settled here.

## 8 Conclusion

Everything above except Conjecture 5 is elementary: each statement is checked by unfolding a definition, and none of it needs dependent types, fibrational machinery, or a reformulation of the notion of partial compiler. The compilers are the ones already written; the relation is what was thrown away when the notion was extracted, and putting it back, read as refutation, gives correctness, a criterion for duplication, a cost model, and a candidate type of verified passes.

The system that prompted the notion has gone: DryadLINQ is no longer maintained, and the cluster it targeted is not the machine anyone compiles for now. The shape of the problem has not gone. Splitting a program, handing the pieces to compilers that know about different targets, and reassembling what they return is the ordinary architecture for heterogeneous and distributed compilation — nested intermediate representations lowered one into the next, machine-learning graphs divided between accelerator and host backends, query plans compiled in stages. What such an architecture does not express, and what is instead maintained outside the types in test suites and comments, is the claim that a stage introduces no errors of its own. That claim is (3), and it costs one relation.

Five things are open, and all are named above: the ordering of Remark 4, for which dropping the symmetry of  $\otimes$  and admitting two residuals, as in the Lambek setting, looks like the way out; the disagreement between the reading of §4 and the monoidal structure, recorded in the footnote to §5; the functoriality of the comprehension (13), which holds only for passes that reflect correctness as well as preserve it; the quotient of Remark 6; and Conjecture 5 itself.

## References

- [1] M. Budiù, J. Galenson and G. D. Plotkin. The compiler forest. In *ESOP 2013*, LNCS 7792, pages 21–40. Springer, 2013.
- [2] V. de Paiva. The Dialectica categories. In *Categories in Computer Science and Logic*, Contemporary Mathematics 92, pages 47–62. American Mathematical Society, Providence, RI, 1989.
- [3] V. de Paiva. *The Dialectica categories*. Technical Report 213, University of Cambridge Computer Laboratory, 1991.
- [4] F. W. Lawvere. Metric spaces, generalized logic, and closed categories. *Rendiconti del Seminario Matematico e Fisico di Milano*, 43:135–166, 1973. Reprinted in *Reprints in Theory and Applications of Categories*, 1:1–37, 2002.
- [5] R. Atkey. Syntax and semantics of quantitative type theory. In *LICS 2018*. ACM, 2018.
- [6] R. Mardare, P. Panangaden and G. D. Plotkin. Quantitative algebraic reasoning. In *LICS 2016*. ACM, 2016.
- [7] V. de Paiva. Partial compilers and Dialectica spaces. Talk at BACAT, September 2011. <https://www.slideshare.net/slideshow/partial-compilers/11320612>

- [8] J. Hedges. Lenses for philosophers. Blog post, 16 August 2018.  
<https://julesh.com/posts/2018-08-16-lenses-for-philosophers.html>
- [9] J. Hedges. The game semantics of game theory. arXiv:1904.11287, 2019.
- [10] V. de Paiva and S. G. da Silva. Kolmogorov-Veloso problems and Dialectica categories.  
arXiv:2107.07854, 2021.